

Dokumentation

Name: Leonard Mottke
Matrikelnummer: 30904
Studiengang: Minf
Prüfungsnummer: 24235

Ausgewählte Themen der Programmierung
Android JetPack/ Xml-Constraint-Layouts mit Kotlin
SoSe 2024

Inhaltsverzeichnis

1	Einleitung.....	4
2	Grundlagen / Technologien	5
2.1	Aufbau einer App.....	5
2.2	Layout	5
2.2.1	Row.....	5
2.2.2	Column	5
2.3	Persistentes Speichern (Optional).....	5
2.3.1	DataOutputStream	5
2.3.2	DataInputStream.....	5
3	Konzept.....	6
3.1	Skizzen	6
3.2	Überblick über die Views	7
3.2.1	Parameterübergabe	8
3.3	Persistente Datenhaltung.....	8
4	Realisierung / Klassen etc.....	9
4.1	Klassen.....	9
4.2	Layouts.....	11
4.2.1	Ablauf	13
4.2.2	Speicherung (optional)	15
5	Zusammenfassung und Erweiterungen.....	17
5.1	Erweiterungen	17
6	Indexverzeichnis	18

Abbildungsverzeichnis

Abbildung 3	Hauptview	8
Abbildung 4	Eingabedialog	8

1 Einleitung

- Hier sollte die Problembeschreibung und Motivation stehen.
- Welche Idee steckt hinter dem Programm

Meine App-Idee war es, eine Anwendung zu entwickeln, mit der man Bücher ausleihen, die Informationen über diese Bücher einsehen und sie anschließend zurückgeben kann. Zu Beginn wusste ich noch nicht genau, wie ich diese Idee technisch umsetzen sollte. Deshalb begann ich damit, einige Skizzen zu zeichnen und einen Plan zu erstellen, wie die App aussehen sollte. Es sollte auch die Möglichkeit geben, ein Buch hinzuzufügen und es nach den besten Bewertungen sortieren zu können.

Ich habe mich dann im Internet nach Inspiration umgesehen, um herauszufinden, wie man am besten eine Android-App entwickelt. Den Code, den wir in den Vorlesungen besprochen haben, habe ich mir erneut angeschaut, aber ich habe mich entschieden, eine andere Variante zu nutzen, um meinen Code zu schreiben. Für mich war es einfacher, die App auf diese Weise zu programmieren und mich intensiver mit Kotlin auseinanderzusetzen.

Beim Erstellen der App gab es einige Herausforderungen, insbesondere die Einarbeitung in neue Konzepte. Das Layout mit XML zu erstellen, war schließlich recht einfach, und es sah auch ansprechend aus. Die größte Herausforderung war jedoch das Schreiben des Codes, da man bei kleineren Fehlern stundenlange Recherchen benötigt, um diese zu lösen.

2 Grundlagen / Technologien

Der folgende Teil konnte umgangen werden durch Xml-Constraint-Layouts eine bessere Variante für meine Android App.

2.1 Aufbau einer App

2.2 Layout

2.2.1 Row

2.2.2 Column

2.3 Persistentes Speichern (Optional)

2.3.1 DataOutputStream

2.3.2 DataInputStream

3 Konzept

3.1 Skizzen

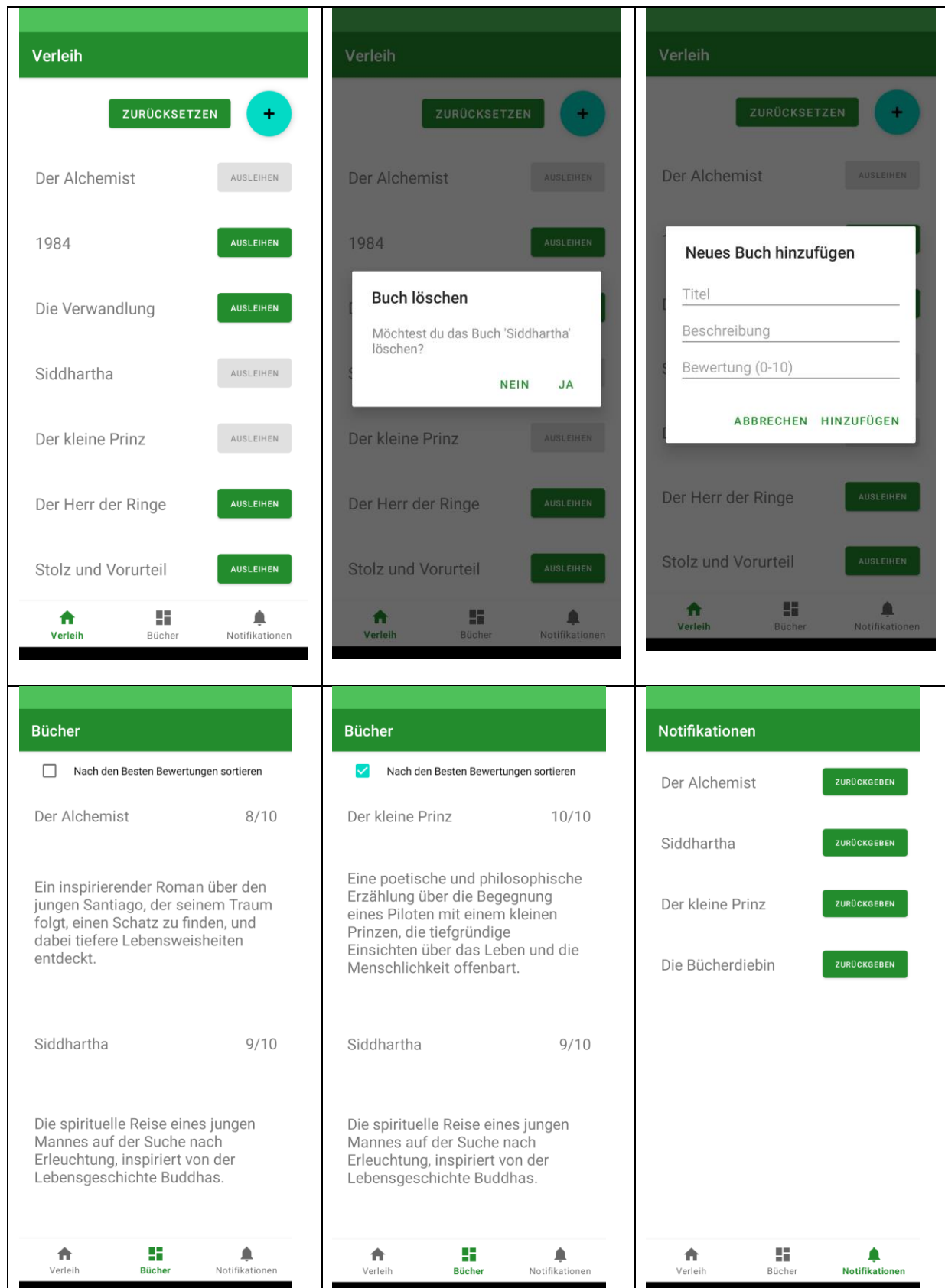
In diesem Kapitel werden einige Skizzen bezüglich der App-Gestaltung dargestellt und beschrieben. Ich habe mich für ein Grün entschieden und Versucht es schön dazustellen.

1) Abbildung	2) Abbildung	3) Abbildung
Das ist die Hauptseite	Die Info über die Bücher	Seite wo man die Bücher Zurück geben kann.

Folgende UI-Elemente sind auf dem Startbildschirm zu sehen:

- Actionbar mit weiteren Schalter
 - Man kann sie löschen, wenn man sie lange hält.
 - Einfügen durch das Plus
 - Ausleihen
 - Etc.
- ListView
 - Aufgelisteten Bücher
 - Am rechten Rand der Zelle kann der Ausleih Button positioniert werden.
- Neben dem Hauptview existieren noch ein zwei weitere Views.
 - Hier werden die verschiedenen Daten mittels UI-Elemente dargestellt.
 - Es muss ja nicht immer eine Eingabezeile sein.

3.2 Überblick über die Views



3.2.1 Parameterübergabe

Ich habe hier verschiedene Möglichkeiten beschrieben, wie ich sie in der App umgesetzt habe.

- **Fragments:** In den **Fragments** wie **HomeFragment** oder **DashboardFragment** gibt es oft die Notwendigkeit, Parameter oder Daten an andere Komponenten wie das **ViewModel** oder **Dialoge** weiterzugeben. Zum Beispiel, wenn im **HomeFragment** ein Buch ausgeliehen oder gelöscht wird, wird das Buchobjekt als Parameter an Methoden wie `homeViewModel.update(buch)` oder `showDeleteBuchDialog(buch)` übergeben. Hier wird das Buchobjekt direkt als Parameter in die entsprechenden Funktionen übergeben, um sicherzustellen, dass die richtigen Daten in der Datenbank aktualisiert oder gelöscht werden.
- **Adapter:** In der **BuchAdapter**-Klasse werden ebenfalls Parameter übergeben, z.B. wenn der Benutzer eine Aktion auf einem Buch ausführt (etwa das Ausleihen). Über die Adapter-Methoden wie `buttonClick = { buch -> homeViewModel.update(buch.copy(istAusgeliehen = true)) }` wird das Buchobjekt an die entsprechende Methode weitergereicht, um den Status zu ändern. Hier zeigt sich die Bedeutung der korrekten Parameterübergabe für das Verhalten der App.

Die Parameterübergabe ist also eine fundamentale Technik, um sicherzustellen, dass die richtigen Daten an die richtigen Stellen weitergegeben werden und die App korrekt funktioniert. Dabei wird oft direkt mit Datenobjekten (wie einem **Buch**-Objekt) gearbeitet, die als Parameter übergeben werden.

3.3 Persistente Datenhaltung

Persistente Datenhaltung wird durch die Verwendung von **Room** und **SQLite** gewährleistet. Hier sorgt die **BuchDatenbank** dafür, dass alle Bücher dauerhaft gespeichert werden, sodass sie auch nach dem Neustart der App zur Verfügung stehen. Die Integration von **Room** mit **DAO** macht den Umgang mit persistenter Datenhaltung in der Android-Entwicklung viel einfacher und strukturierter.

4 Realisierung / Klassen etc

4.1 Klassen

Beispiel Klasse Home Fragment:

Die Klasse HomeFragment ist für die Anzeige und Verwaltung der Bücher zuständig. Sie ist ein Fragment, das die Hauptfunktionen der App unterstützt, wie das Hinzufügen, Löschen und Verwalten von Büchern.

```
package com.example.buecherverlei.ui.home

import ...

// Fragment zur Anzeige und Verwaltung von Büchern
class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!
    private lateinit var adapter: BuchAdapter
    private val homeViewModel: HomeViewModel by viewModels()

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, attachToParent: false)
        val root: View = binding.root

        // RecyclerView initialisieren
        homeViewModel.initialisierung()
        binding.recyclerView.layoutManager = LinearLayoutManager(requireContext())
        adapter = BuchAdapter(
            buttonClick = { buch -> homeViewModel.update(buch.copy(istAusgeliehen = true)) },
            sektionHalten = { buch -> showDeleteBuchDialog(buch) }
        )
        binding.recyclerView.adapter = adapter
    }
}
```

Buchliste anzeigen:

- Die RecyclerView zeigt alle Bücher an, die in der App gespeichert sind. Dabei wird ein Adapter (BuchAdapter) verwendet, um die Daten aus dem HomeViewModel darzustellen. Der ViewModel sorgt dafür, dass die Daten immer aktuell bleiben, und die Liste automatisch aktualisiert wird.

```

// FloatingActionButton zum Hinzufügen neuer Bücher
binding.floatingActionButton.setOnClickListener { it: View!
    showAddBookDialog(homeViewModel)
}

// Button zum Zurücksetzen der Datenbank
binding.buttonZuruecksetzen.setOnClickListener { it: View!
    homeViewModel.resetDatabase()
    homeViewModel.insertBuecher(AnfangBuecherListe.buecherliste)
}

// Beobachtet und aktualisiert die Buchliste im Adapter
homeViewModel.buecherList.observe(viewLifecycleOwner, Observer { buchlist ->
    adapter.submitList(buchlist)
})

return root
}

```

Bücher hinzufügen:

- Über den FloatingActionButton kann man ein neues Buch hinzufügen. Wenn man auf den Button klickt, öffnet sich ein Dialog, in dem man den Titel, die Beschreibung und die Bewertung des Buches eingeben kann. Diese Eingaben werden überprüft und nur wenn alle gültig sind, wird das Buch zur Datenbank hinzugefügt. Das hat mir geholfen, ein besseres Verständnis für Benutzerinteraktionen und das Arbeiten mit Dialogen in Android zu bekommen.

Bücher löschen:

- Man kann ein Buch durch langes Drücken auf das Element in der Liste löschen. Es öffnet sich dann ein Dialog, der eine Bestätigung abfragt, ob man das Buch wirklich entfernen möchte. Diese Funktion war für mich hilfreich, um zu verstehen, wie man Bestätigungsdialoge implementiert und Daten in der App verwaltet.

Datenbank zurücksetzen:

- Es gibt eine Schaltfläche, die es ermöglicht, die Datenbank zurückzusetzen und die Bücherliste auf einen Ausgangszustand mit einer vordefinierten Liste von Büchern zurückzusetzen. Hier habe ich gelernt, wie man Datenbanken in Android mit ViewModels verwaltet und initialisiert.

```

// Zeigt einen Dialog zur Eingabe neuer Buchdaten an
private fun showAddBookDialog(homeViewModel: HomeViewModel) {
    val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_add_book, root: null)
    val titelEditText: EditText = dialogView.findViewById(R.id.EditTextTitel)
    val beschreibungEditText: EditText = dialogView.findViewById(R.id.etBeschreibung)
    val bewertungEditText: EditText = dialogView.findViewById(R.id.etBewertung)

    AlertDialog.Builder(requireContext()) AlertDialog.Builder
        .setTitle("Neues Buch hinzufügen") AlertDialog.Builder!
        .setView(dialogView)
        .setPositiveButton(text: "Hinzufügen") { _, _ ->
            val titel = titelEditText.text.toString()
            val beschreibung = beschreibungEditText.text.toString()
            val bewertung = bewertungEditText.text.toString().toIntOrNull()

            // Eingaben validieren und Buch hinzufügen
            if (titel.isNotEmpty() && beschreibung.isNotEmpty() && bewertung != null && bewertung in 0..10) {
                val neuesBuch = Buch(titel, beschreibung, bewertung)
                homeViewModel.insertBuch(neuesBuch)
            }
        }
        .setNegativeButton(text: "Abbrechen", listener: null)
        .create() AlertDialog!
        .show()
}

// Zeigt einen Dialog zur Bestätigung des Löschens eines Buches an
private fun showDeleteBuchDialog(buch: Buch) {
    val linearLayout = LinearLayout(requireContext()).apply { this: LinearLayout
        orientation = LinearLayout.VERTICAL
        setPadding(left: 50, top: 40, right: 50, bottom: 10)
    }

    val message = TextView(requireContext()).apply { this: TextView
        text = "Möchtest du das Buch '${buch.titel}' löschen?"
        textSize = 16f
        setTextColor(Color.GRAY)
        textAlignment = View.TEXT_ALIGNMENT_TEXT_START
    }

    linearLayout.addView(message)

    AlertDialog.Builder(requireContext()) AlertDialog.Builder
        .setTitle("Buch löschen") AlertDialog.Builder!
        .setView(linearLayout)
        .setPositiveButton(text: "Ja") { dialog, _ ->
            homeViewModel.delete(buch)
            dialog.dismiss()
        }
        .setNegativeButton(text: "Nein") { dialog, _ ->
            dialog.cancel()
        }
        .show()
}

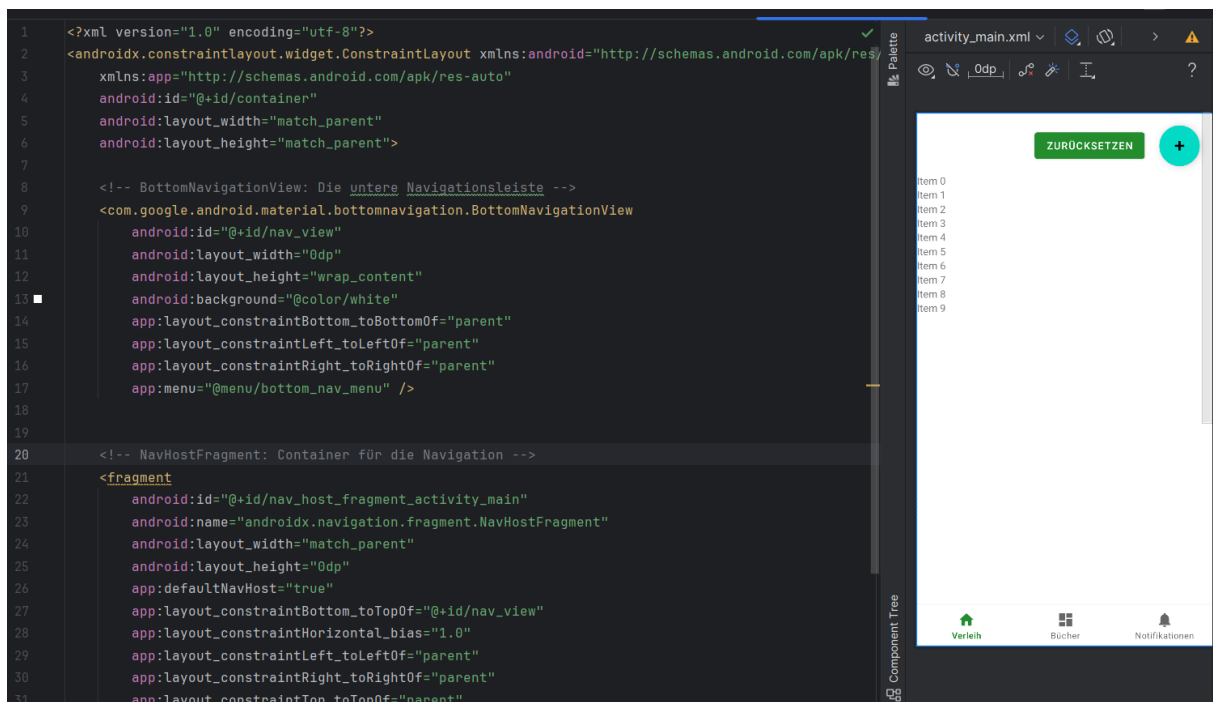
```

Lifecycle-Management:

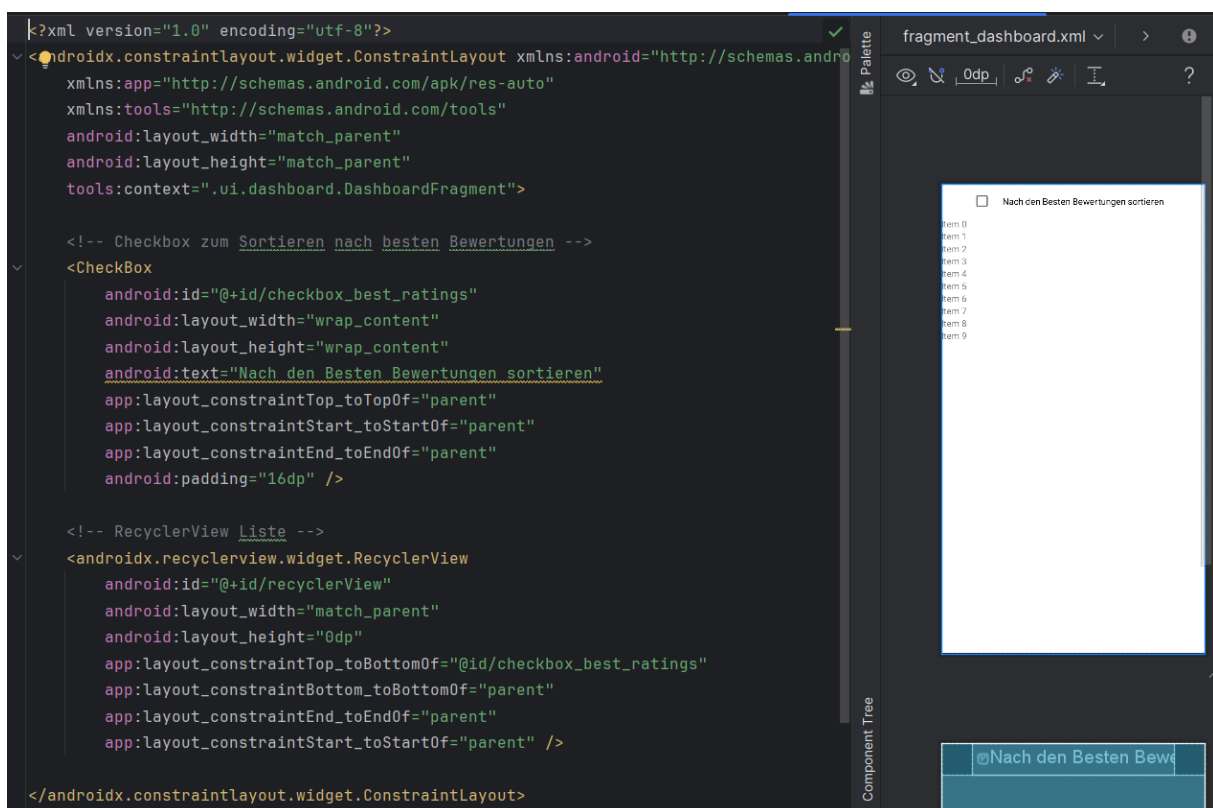
- Die Klasse beachtet den Lebenszyklus des Fragments, indem sie im `onDestroyView()` die Binding-Referenz auf null setzt. Dadurch werden Speicherlecks verhindert. Das war ein wichtiger Punkt, um zu verstehen, wie man sauber mit Ressourcen umgeht.

4.2 Layouts

Beispiel xml Code für layouts:



Dieses XML-Layout beschreibt die Benutzeroberfläche einer Aktivität, die sowohl eine **BottomNavigationView** als auch einen **NavHostFragment** enthält, um die Navigation innerhalb der App zu ermöglichen.



Erklärungen zum Layout:

Root-Layout:

- Das gesamte Layout ist in einem `ConstraintLayout` enthalten, was eine flexible Anordnung der UI-Elemente ermöglicht. Es stellt sicher, dass alle Elemente im Verhältnis zueinander und zum übergeordneten Layout positioniert werden.

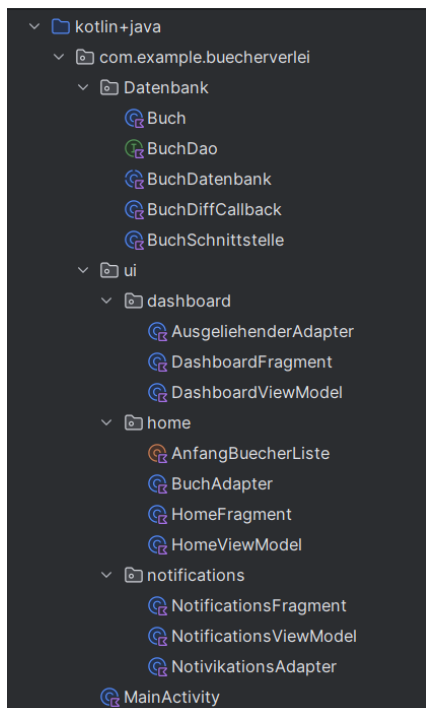
Checkbox für Sortierung:

- **CheckBox:** Diese Checkbox mit der ID `checkbox_best_ratings` ermöglicht es dem Benutzer, die Liste der Bücher nach den besten Bewertungen zu sortieren.
- Die Beschriftung lautet: „Nach den Besten Bewertungen sortieren“.
- Die Position der Checkbox wird durch Constraints festgelegt: Sie ist oben (zur Oberkante des übergeordneten Layouts) und mittig ausgerichtet (durch Constraints zu Start und Ende des übergeordneten Layouts).

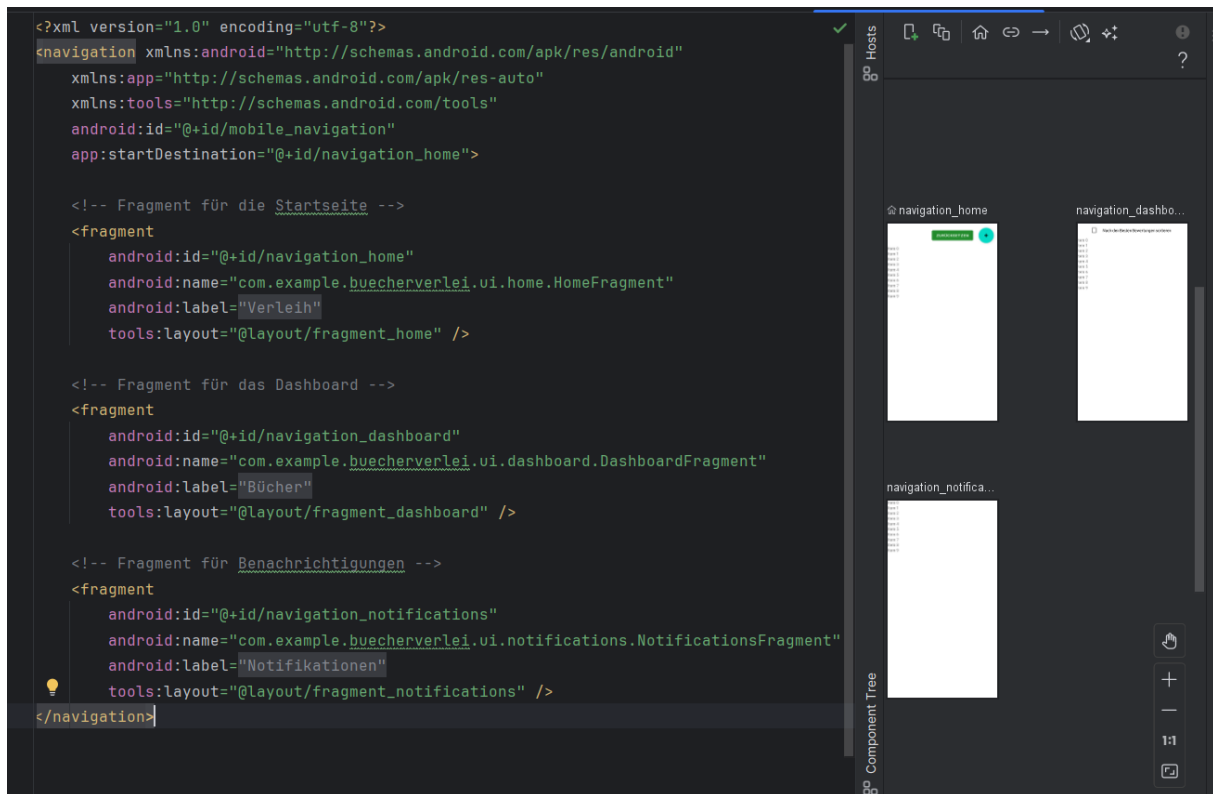
RecyclerView für die Buchliste:

- **RecyclerView:** Darunter befindet sich eine RecyclerView mit der ID `recyclerView`, in der die Liste der Bücher angezeigt wird.
- Diese RecyclerView ist über eine Constraint-Verbindung an die Checkbox gebunden (`app:layout_constraintTop_toBottomOf="@id/checkbox_best_ratings"`), sodass sie direkt unter der Checkbox erscheint.
- Die Höhe der RecyclerView ist auf `0dp` gesetzt, was in Kombination mit den Constraints bedeutet, dass die RecyclerView den verbleibenden Platz einnimmt, der von der Checkbox übrig bleibt.

4.2.1 Ablauf



Navigation:



Dieser XML-Code definiert die Navigationsstruktur der Bücherverleih-App. Es wird verwendet, um zwischen verschiedenen Bereichen der App zu navigieren. Die Navigation erfolgt zwischen verschiedenen Fragments, die jeweils für unterschiedliche Funktionen zuständig sind.

Grundstruktur:

- Die Navigation wird durch das `<navigation>`-Element definiert. Dies ist eine zentrale XML-Datei für die Navigation in Android, die den Wechsel zwischen verschiedenen Bildschirmen (Fragments) der App ermöglicht.

Startziel (startDestination):

- Die Startseite der App wird durch das Attribut `app:startDestination` definiert. In diesem Fall ist die Startseite das Fragment mit der ID `navigation_home`, welches das `HomeFragment` lädt. Dies bedeutet, dass die App beim Starten zuerst dieses Fragment anzeigt.

Fragment für die Startseite (HomeFragment):

- Dieses Fragment zeigt die Bücher, die ausgeliehen werden können. Es ist unter der ID `navigation_home` registriert, und seine Layout-Datei ist `fragment_home`. Dieses Fragment ist das erste, das dem Nutzer angezeigt wird, wenn die App gestartet wird.

Fragment für das Dashboard (DashboardFragment):

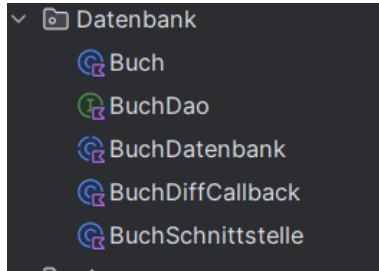
- Das Fragment mit der ID `navigation_dashboard` repräsentiert das Dashboard. Es könnte Informationen über die verfügbaren Bücher oder Statistiken anzeigen. Das zugehörige Layout ist `fragment_dashboard`.

Fragment für Benachrichtigungen (NotificationsFragment):

- Dieses Fragment zeigt Benachrichtigungen an und ist unter der ID `navigation_notifications` gespeichert. Das zugehörige Layout ist `fragment_notifications`.

4.2.2 Speicherung (optional)

Datenbank:



In der Datenbankstruktur der App, wie im Bild gezeigt, ist eine SQLite-Datenbank für die Verwaltung von Büchern integriert. Diese besteht aus mehreren wichtigen Komponenten, die zusammenarbeiten, um die Speicherung und Verwaltung der Daten effizient zu gestalten.

Überblick über die Klassen in der Datenbank:

Buch:

- **Funktion:** Die Klasse **Buch** repräsentiert ein Datenmodell für ein Buch in der App. Sie enthält die wesentlichen Informationen über ein Buch, wie Titel, Beschreibung und Bewertung. Diese Klasse wird verwendet, um die Bücher als Objekte in der Datenbank zu speichern und darzustellen.
- **Datenstruktur:** Typischerweise enthält die Klasse Felder, die den Attributen eines Buchs entsprechen, und sie ist mit Annotations wie `@Entity` versehen, um sie als Datenbanktabelle zu kennzeichnen.

BuchDao:

- **Funktion:** **DAO** steht für **Data Access Object**. Diese Schnittstelle enthält Methoden zur Interaktion mit der Datenbank, wie das Einfügen, Aktualisieren, Löschen oder Abfragen von Büchern.
- **Beispielmethode:**
 - `insert(Buch)`: Fügt ein Buch in die Datenbank ein.
 - `delete(Buch)`: Entfernt ein Buch aus der Datenbank.
 - `getAllBooks()`: Ruft alle Bücher aus der Datenbank ab.

BuchDatenbank:

- **Funktion:** Diese Klasse verwaltet die eigentliche Datenbank. Sie definiert die Verbindung zur Datenbank, die Tabellen (in diesem Fall die "Buch"-Tabelle) und stellt die DAO-Methoden bereit. Sie ist in der Regel mit `RoomDatabase` verbunden und sorgt dafür, dass die Datenbank richtig initialisiert und konfiguriert wird.
- **Singleton-Prinzip:** Die Datenbank wird meist als Singleton implementiert, um sicherzustellen, dass nur eine Instanz der Datenbank gleichzeitig im gesamten Anwendungskontext verwendet wird.

BuchDiffCallback:

- **Funktion:** Diese Klasse ist für die Vergleichslogik verantwortlich, die prüft, ob zwei Buchobjekte gleich sind. Sie wird von der RecyclerView verwendet, um effizient Änderungen in der Liste der Bücher zu erkennen und nur die geänderten Einträge zu aktualisieren. Dies verbessert die Performance der App, insbesondere bei großen Datenmengen.
- **DiffUtil.Callback:** Durch die Implementierung von `DiffUtil.Callback` kann die RecyclerView effizient arbeiten und dynamisch Buchobjekte aktualisieren, löschen oder hinzufügen.

BuchSchnittstelle:

- **Funktion:** Diese Klasse (vermutlich eine Schnittstelle) stellt Methoden bereit, um die Interaktionen mit der Buchdatenbank auf höherer Ebene zu abstrahieren. Sie könnte beispielsweise für die Kommunikation zwischen den verschiedenen Schichten der App verwendet werden, wie z.B. der Verbindung zwischen dem ViewModel und der DAO.

Was die Datenbank insgesamt macht:

Diese Datenbankstruktur ermöglicht es der App, Bücher zu verwalten – d.h., Bücher hinzuzufügen, zu löschen und abzufragen. Durch die Verwendung des DAO-Musters wird sichergestellt, dass alle Datenbankoperationen klar definiert und gut organisiert sind. Die BuchDatenbank-Klasse sorgt für eine ordnungsgemäße Verwaltung und Initialisierung der Datenbank, während BuchDiffCallback die Benutzeroberfläche flüssig und effizient hält.

Hier ein Code Beispiel BuchDatenbank:

```
package com.example.buecherverlei.Datenbank

> import ...

@Database(entities = [Buch::class], version = 1, exportSchema = false)
abstract class BuchDatenbank : RoomDatabase() {

    // Liefert das DAO für die Datenbankoperationen
    abstract fun buchDao(): BuchDao

    companion object {
        // Enthält die Singleton-Instanz der Datenbank, um Mehrfach-Instanzen zu vermeiden
        @Volatile
        private var INSTANCE: BuchDatenbank? = null

        // Gibt die Singleton-Instanz der Datenbank zurück oder erstellt sie
        fun getDatabase(context: Context): BuchDatenbank {
            return INSTANCE ?: synchronized(lock: this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    BuchDatenbank::class.java,
                    name: "game_database" // Name der Datenbank
                ).build()
                INSTANCE = instance
                instance ^synchronized
            }
        }
    }
}
```


5 Zusammenfassung und Erweiterungen

Es war ein hartes Stück Arbeit. Das Schwierigste war mit Abstand die Fehlersuche, wenn etwas nicht funktioniert hat oder wenn man offensichtliche Dinge übersehen hat – zum Beispiel ein Komma, das falsch gesetzt war, oder eine Zeile, in der man sich vertan hat. Trotzdem kam man irgendwie zurecht. Es hat mich auf jeden Fall in meinen Programmierkenntnissen weitergebracht.

Ich habe gelernt, wie man eine Datenbank anlegt, was alles hineingehört, und wie man die Klassen so schreibt, dass sie richtig zusammenarbeiten – alles essentielle Dinge für den Beruf.

Abschließend kann man sagen: Ist meine App perfekt? Naja, nein. Aber habe ich das Beste gegeben, was ich mit meinem aktuellen Wissen schaffen konnte? Ja. Damit habe ich eine für mich zufriedenstellende App entwickelt, die alle Teile der Aufgabenstellung erfüllt. Auch wenn es nur das Minimum ist, bin ich zufrieden.

5.1 Erweiterungen

Sinnvolle Erweiterungen wären zum Beispiel:

- Sortierung der Liste nach Namen.
- Sortierung der Liste nach Neusten hinzugefügt.
- Filter nach Namen des Buches.
- Einfacheres Löschen von mehreren Einträgen.
- Synchronisation mit einer Server SQL-Datenbank.
- Mehrfache Buch Eingabe soll nicht möglich sein, keine Dopplungen der Bücher.

6 Indexverzeichnis

Ablauf	10
DataInputStream	6
DataOutputStream	5
Einleitung	4
Klassen	10
Konzept	8
Layout	5
Layouts	10
Parameterübergabe	9
Persistente Datenhaltung	9
DataInputStream	6
DataOutputStream	5
Realisierung	10
Weiterer View	9
Parameterübergabe	9
Zusammenfassung	11

